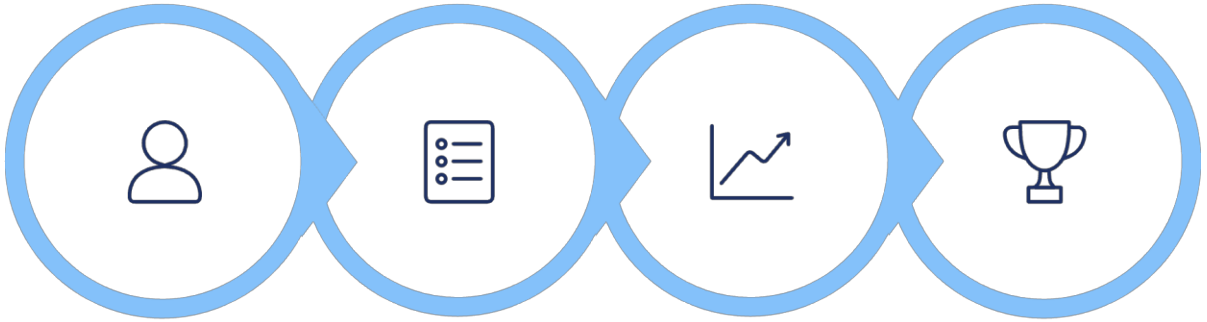
Artificial Intelligence Workshop

Recommender Systems

What a Recommender System Does

A recommender system is an **information-filtering tool** that estimates which items are most relevant to a user or decision context and returns a **ranked list** — not a simple yes/no prediction.

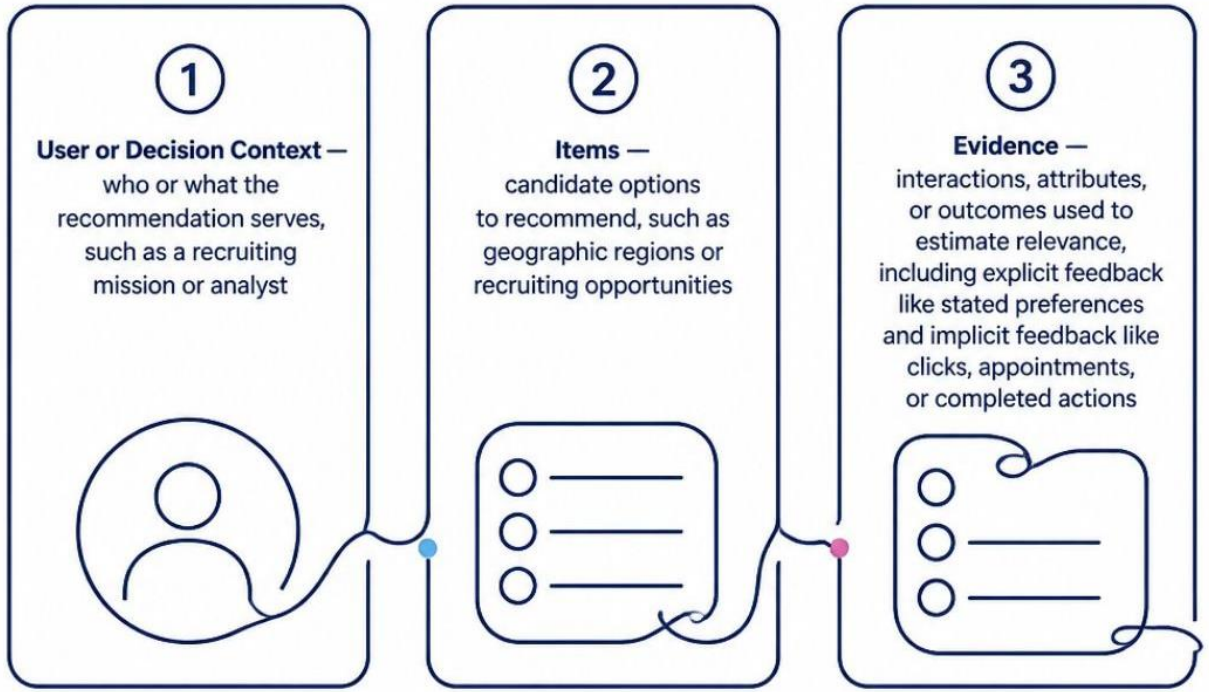


Streaming platforms rank movies; online retailers rank products. For Army recruiting: **mission need** = specified occupation, **items** = geographic markets, **output** = regions warranting analyst review.

A recommender narrows a large option set into a prioritized list.

The Three Building Blocks

Every recommender requires three components to produce meaningful output.



Explicit vs. Implicit Feedback

Explicit: Direct signals — ratings, stated preferences, survey responses.

Implicit: Observed behavior — clicks, views, appointments, completed actions.

In Army recruiting, the **"user"** may be a recruiting mission or analyst — not an individual person.

- The model needs a clearly defined decision context, candidate options, and meaningful evidence.

How Recommendations Are Produced

01

Define the Recommendation Objective

02

Prepare and Validate the Data

03

Represent Users and Items Numerically

04

Calculate Relevance or Similarity

05

Exclude Infeasible or Prohibited Options

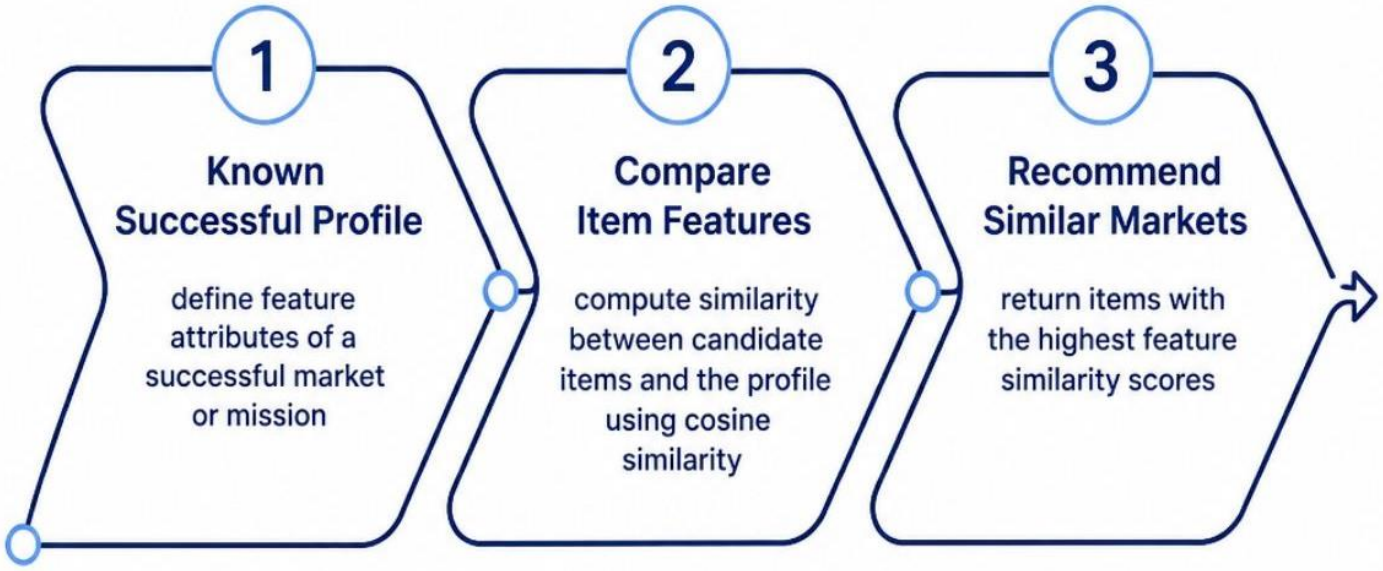
06

Rank and Return Top K Options

Top K refers to the first K recommendations returned. Data quality and requirements come *before* the algorithm.

Recommendations are created by scoring, filtering, and ranking — not by intuition alone.

Content-Based Filtering



How It Works

Recommends items with attributes similar to those previously associated with success. Each item and mission profile is represented by features; similarity is measured (commonly via **cosine similarity**).

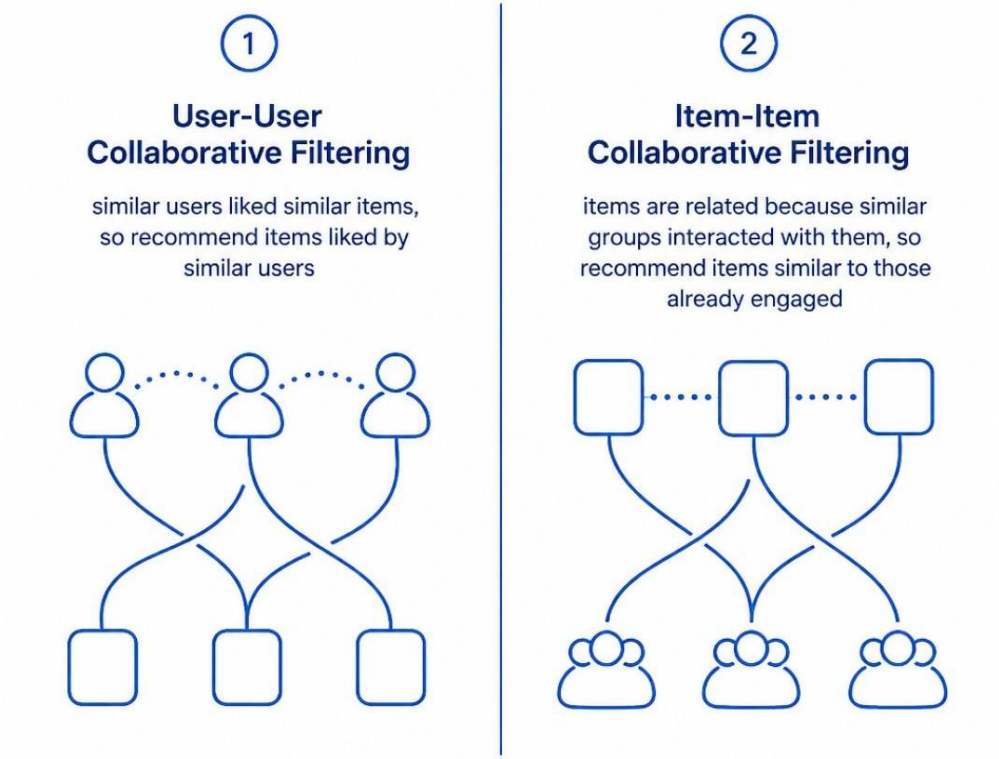
Recruiting example: A region is described by occupational vacancies, education programs, distance, station capacity, labor-market indicators, and historical outcomes.

- **Strength:** Explainable; can recommend new items with no interaction history
- **Limitation:** Limited to attributes in the data; may repeatedly recommend "more of the same"

Content-based systems recommend options that resemble a defined mission profile.

Collaborative Filtering

Learns from collective behavior patterns. Assumes users or items with similar historical interactions may remain related — **without requiring detailed item descriptions**.



Recruiting Translation

Similar recruiting missions, stations, or occupational requirements may have produced similar regional outcomes — even when descriptive labels differ.

- Does not require detailed feature engineering for every region
- Discovers relationships not visible in item descriptions
- **Warning:** Correlation does not imply causation

Collaborative filtering discovers relationships from behavior that may not appear in item descriptions.

Matrix Factorization and Latent Factors

The Problem: Sparse Data

Each user interacts with only a small fraction of available items, leaving most of the interaction matrix empty.

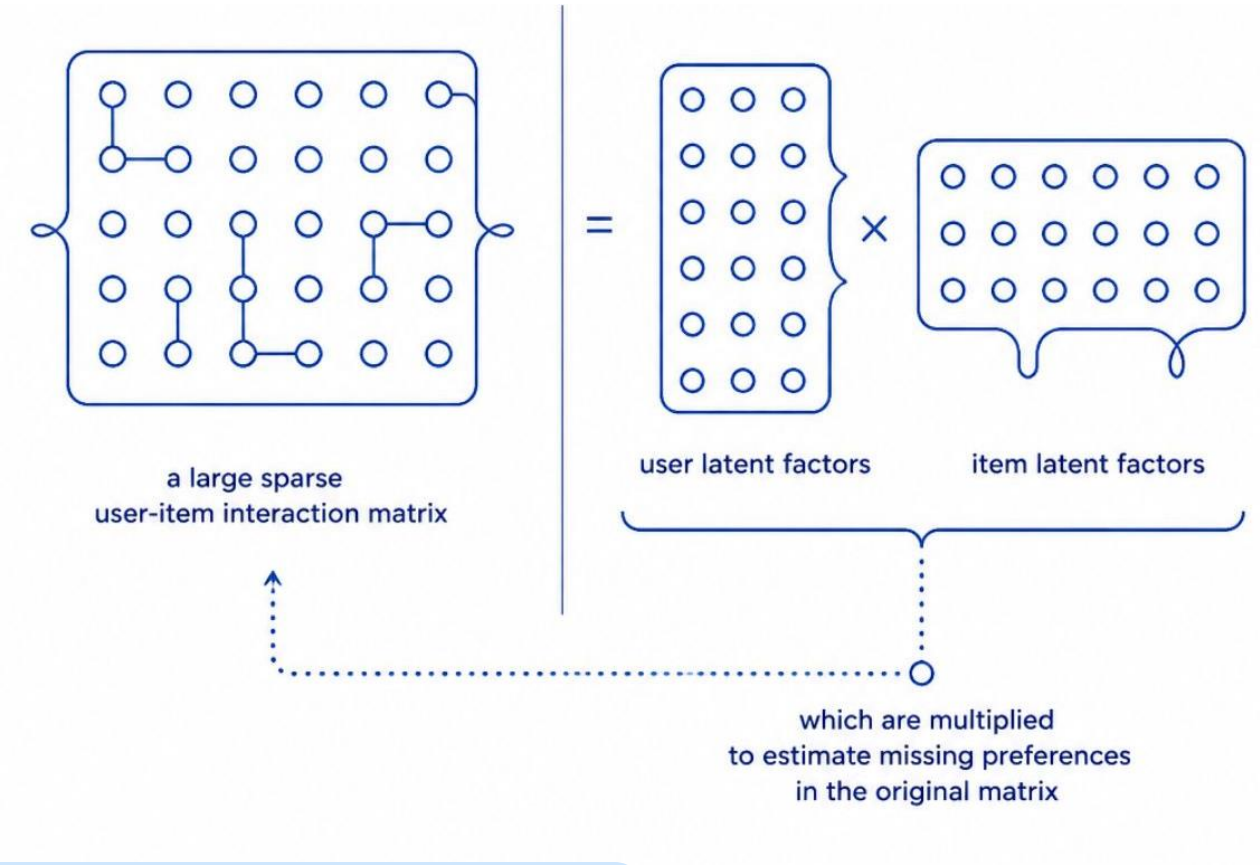
The Solution: Compress and Reveal

Matrix factorization (e.g., SVD) compresses the sparse matrix into **latent factors** — hidden dimensions learned from data.

User factors × Item factors ≈ Predicted preference

- Technical vs. nontechnical orientation
- Local opportunity vs. willingness to travel
- Immediate availability vs. long training timeline

Latent factors may be **difficult to interpret**.



Matrix factorization can uncover hidden preference patterns, but it is less transparent.

Hybrid Recommender Systems

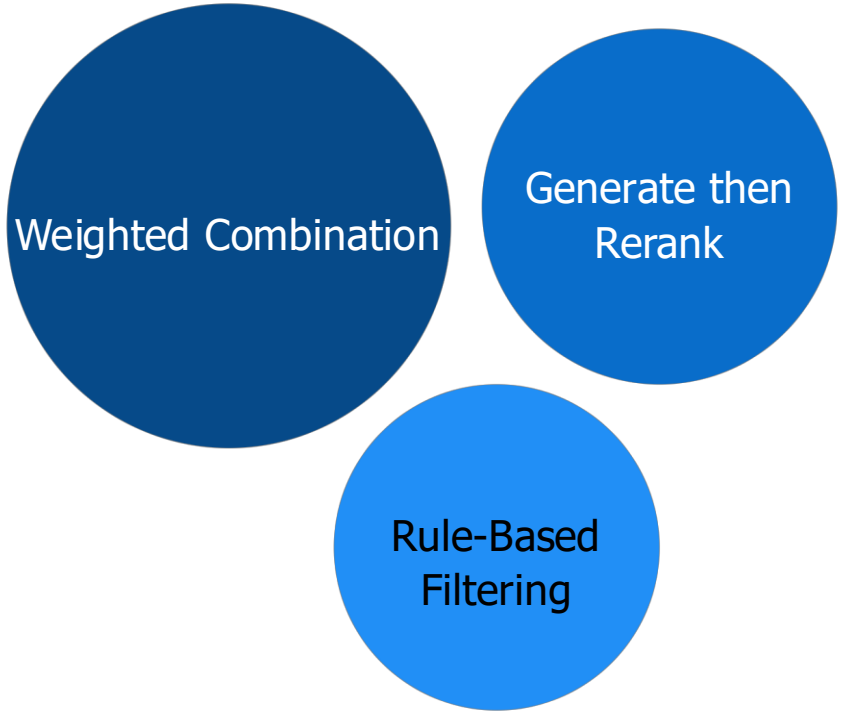
A hybrid recommender combines multiple evidence sources for more robust recommendations.

Content Score
Does the region have relevant education and occupational indicators?

Collaborative Score
Did similar missions perform well in comparable regions?

Mission Constraints
Available position, acceptable distance, sufficient station capacity, current data?

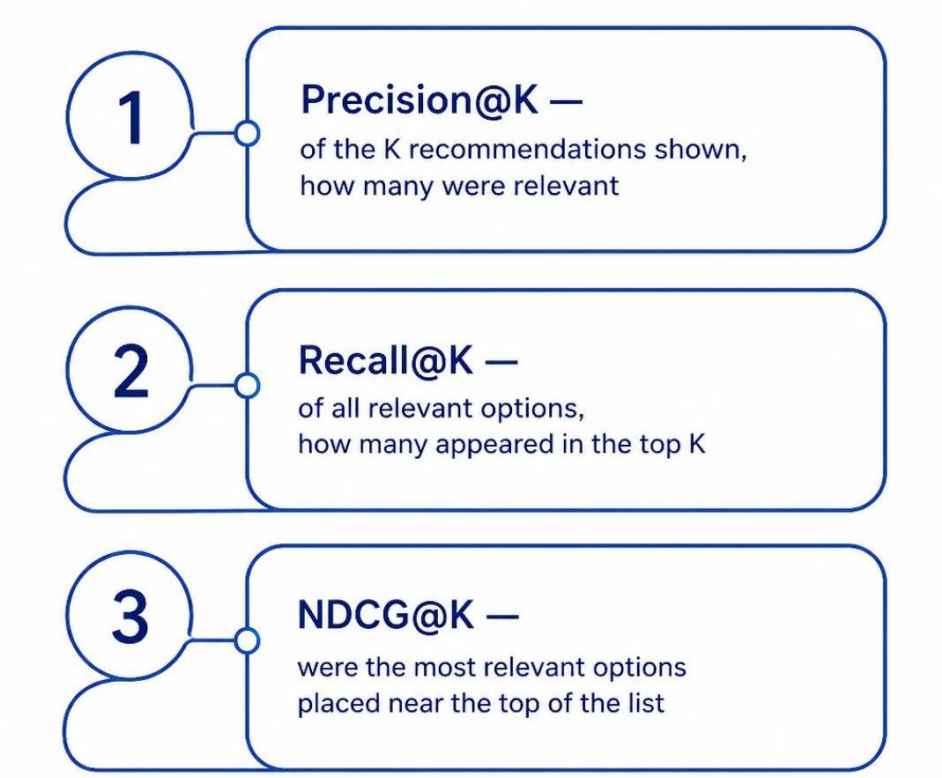
- **Strength:** More robust when one data source is weak
- **Limitation:** More difficult to tune, explain, and govern



Hybrid systems combine complementary evidence but require clear weighting and oversight.

How Recommenders Are Evaluated

Ordinary accuracy is insufficient — the output is a **ranked list**. Evaluation must measure ranking quality and operational value.



Operational Measures

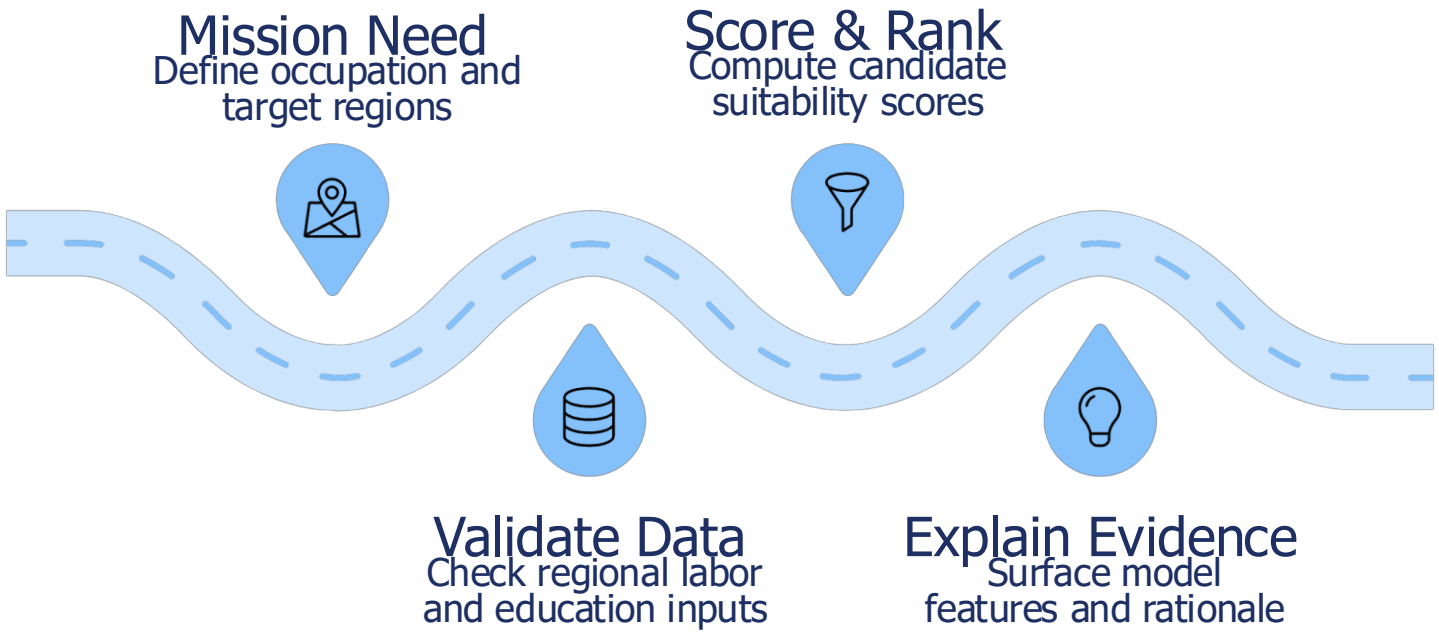
- Analyst acceptance rate
- Time saved per review cycle
- Stability across repeated runs
- Geographic and mission coverage
- Rate of recommendations rejected due to bad or stale data

Relevance labels must be defined by **recruiting domain experts**.

A recommendation is useful only when its ranking quality and operational value are measured.

Adapting the Model to Regional Army Recruiting

Mission: Identify regions warranting additional human analysis for recruiting a specified Army occupation.



Inputs

- Occupational positions, education programs, labor-market indicators
- Historical rates, station capacity, data completeness

Output

- Top three regions with evidence, confidence, and data flags
- Requires human approval before any action

The recruiting recommender prioritizes where analysts should investigate — not whom the Army should recruit.

Responsible Deployment: The R.O.A.D. Model

1. **R — Requirements**
   Define decision, mission, constraints, and error costs.
2. **O — Operationalize Data**
   Standardize definitions, validate records, identify authoritative sources.
3. **A — Analytics**
   Compare content-based, collaborative, factorization, and hybrid approaches.
4. **D — Deployment**
   Present recommendations with monitoring and human approval.

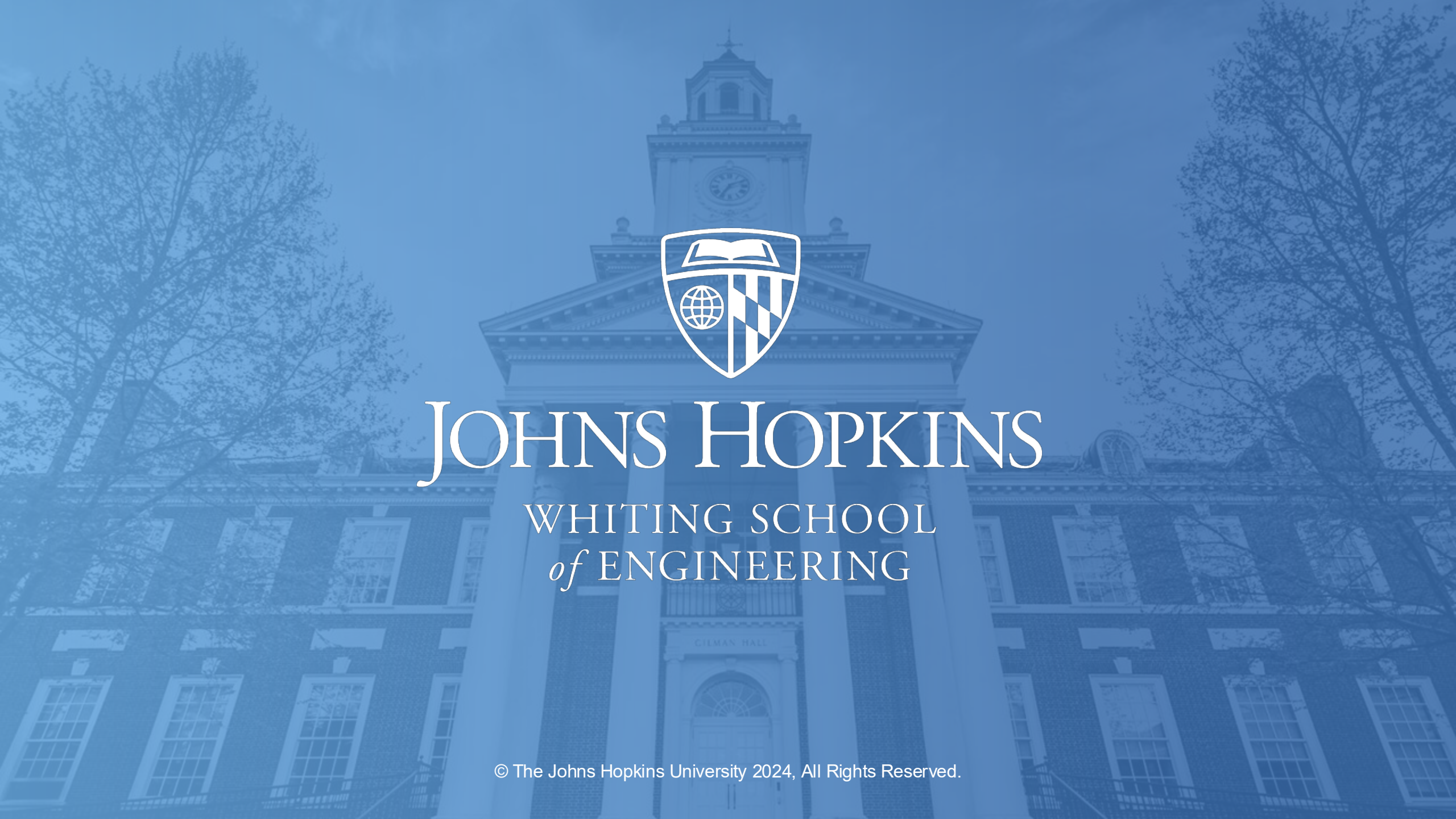
Key Risks

- Incomplete, stale, or duplicated data
- Historical exposure bias and popularity feedback loops
- Correlation misinterpreted as causation
- Protected characteristic influence
- Automated action without human review

Decision Rule: No region should be operationally prioritized solely because the model ranked it highly. A responsible recommender combines clear requirements, consistent data, measured ranking quality, explainable evidence, and human judgment.

Johns Hopkins Whiting School of Engineering

Your Turn



Artificial Intelligence Workshop

Generative AI, Agentic AI, and RAG

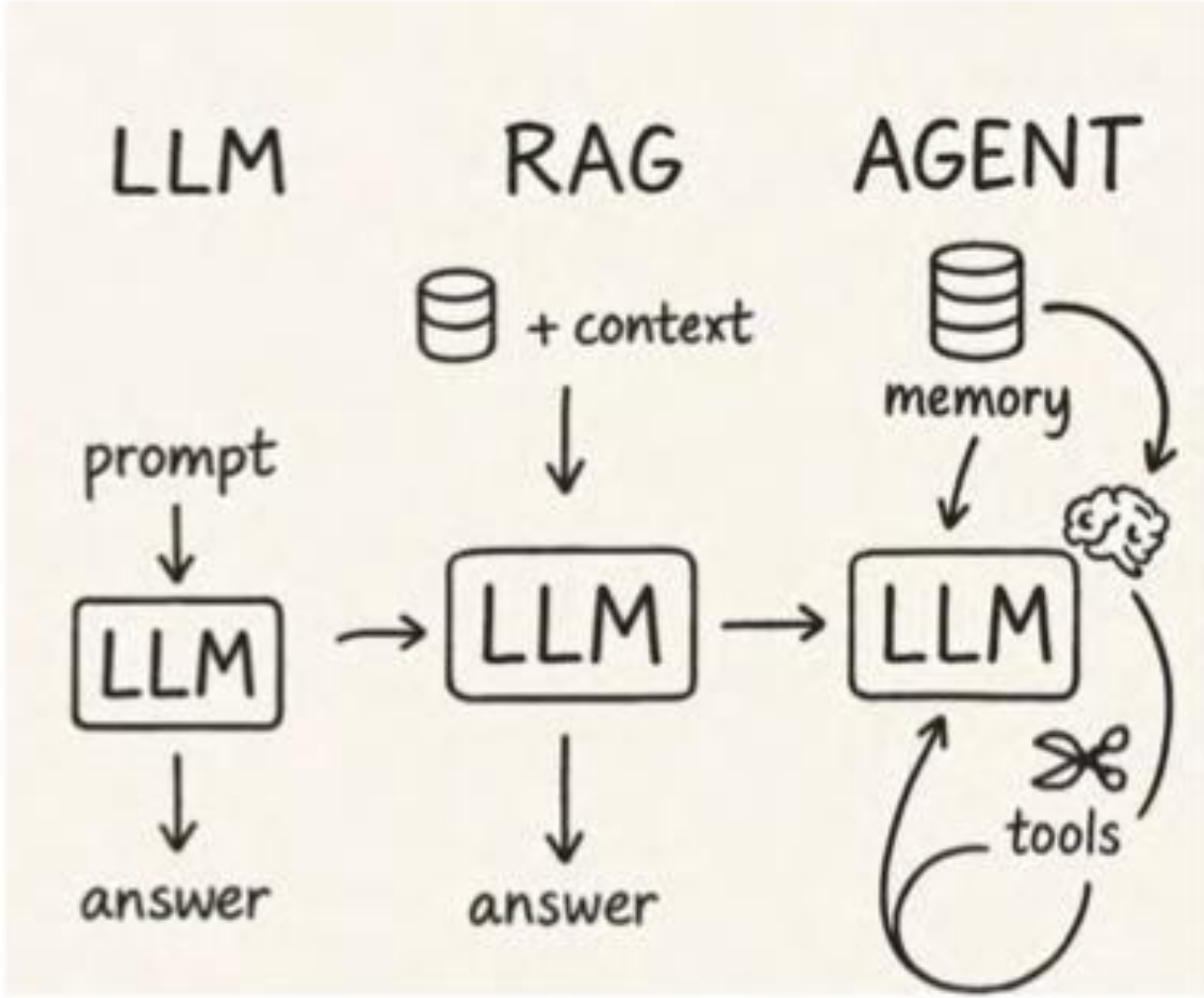
10 Things You Should Know

1. **Foundation Model** – general-purpose pre-trained base.
2. **Large Language Model (LLM)** – specialized for language understanding/generation.
3. **Large Reasoning Model (LRM)** – explicit reasoning capability.
4. **Mixture of Experts (MoE)** – modular scaling with specialized sub-models.
5. **Vector Database** – retrieval infrastructure for embeddings.
6. **RAG** – retrieval-augmented generation for grounded answers.
7. **Model Context Protocol (MCP)** – extends models with tools & APIs.
8. **Agent** – goal-driven AI that perceives, reasons, and acts.
9. **Agentic AI** – the paradigm of AI-native, multi-agent operating models.
10. **Artificial Super Intelligence (ASI)** – machine conciseness - science fiction

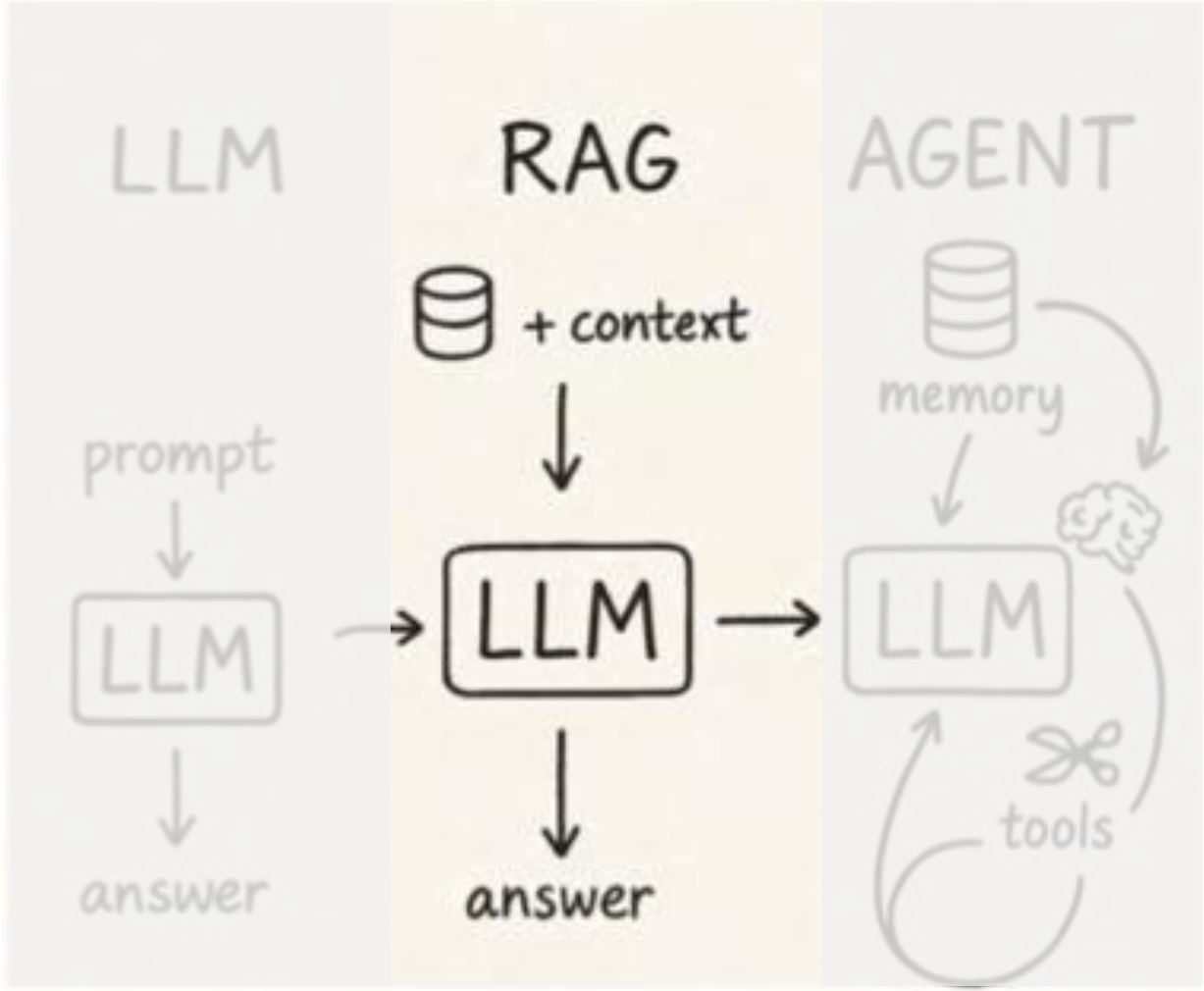
Gen AI – AI Agent – Agentic AI

System Type	GenAI (LLM Only)	AI Agent	Agentic AI
Task Capability	Answers based on pre-trained knowledge	Takes input, decides and completes a task	Handles multi-step goals with planning and coordination
Tool Usage	No external tools access	Uses tools to complete a task	Uses multiple tools, may call other agents
Autonomous Decisions	No decision-making	Makes decision to complete a task	Plans, decides, and adapts over time

LLM – RAG – Agentic AI



LLM – RAG – Agentic AI



Challenges in Retrieval-Based Systems

- **Lack of Synthesis:** They return multiple relevant sources but do not summarize or interpret the data cohesively.

- **User Burden:** Users must manually review and extract insights from retrieved content.

What is RAG?

- **Information Retrieval Systems**

- **Generative Models**

RAG’s Strengths: Why It Stands Out

- **Contextual Accuracy**
- **Factual Grounding**
- **Scalability**
- **Customizability**

The Era Before RAG

- **Retrieval-Only Models**
  - BM25, TF-IDF, embedding-based retrieval

- **Generative-Only Models**
  - GPT-3, GPT-3.5, GPT-4, Other LLMs

What Makes RAG Different?

- **Live, up-to-date knowledge integration**
- **More context-aware, accurate responses**
- **A reduction in hallucinated outputs**

RAG (A Unified Approach)

- **Uses retrieval (BERT-like)**

- **Uses generation (GPT-like)**

RAG (A Unified Approach)

- **Uses retrieval (BERT-like)**
- **Uses generation (GPT-like)**
- **Strengths:**
  - Retrieves live, up-to-date data
  - Generates coherent responses
  - Handles complex, context-dependent questions

Advantages of RAG

- **RAG bridges the gap by retrieving AND summarizing**

Query: *"What happens in a breach of contract?"*

- **Retrieval-Only Output:**
  - "Here are five articles on breach of contract." *(User has to read all of them.)*
- **RAG Output:**
  - "A breach of contract occurs when one party fails to fulfill their obligations. Common causes include non-performance or delayed performance, as outlined in these legal sources."
    (A synthesized and meaningful answer!)

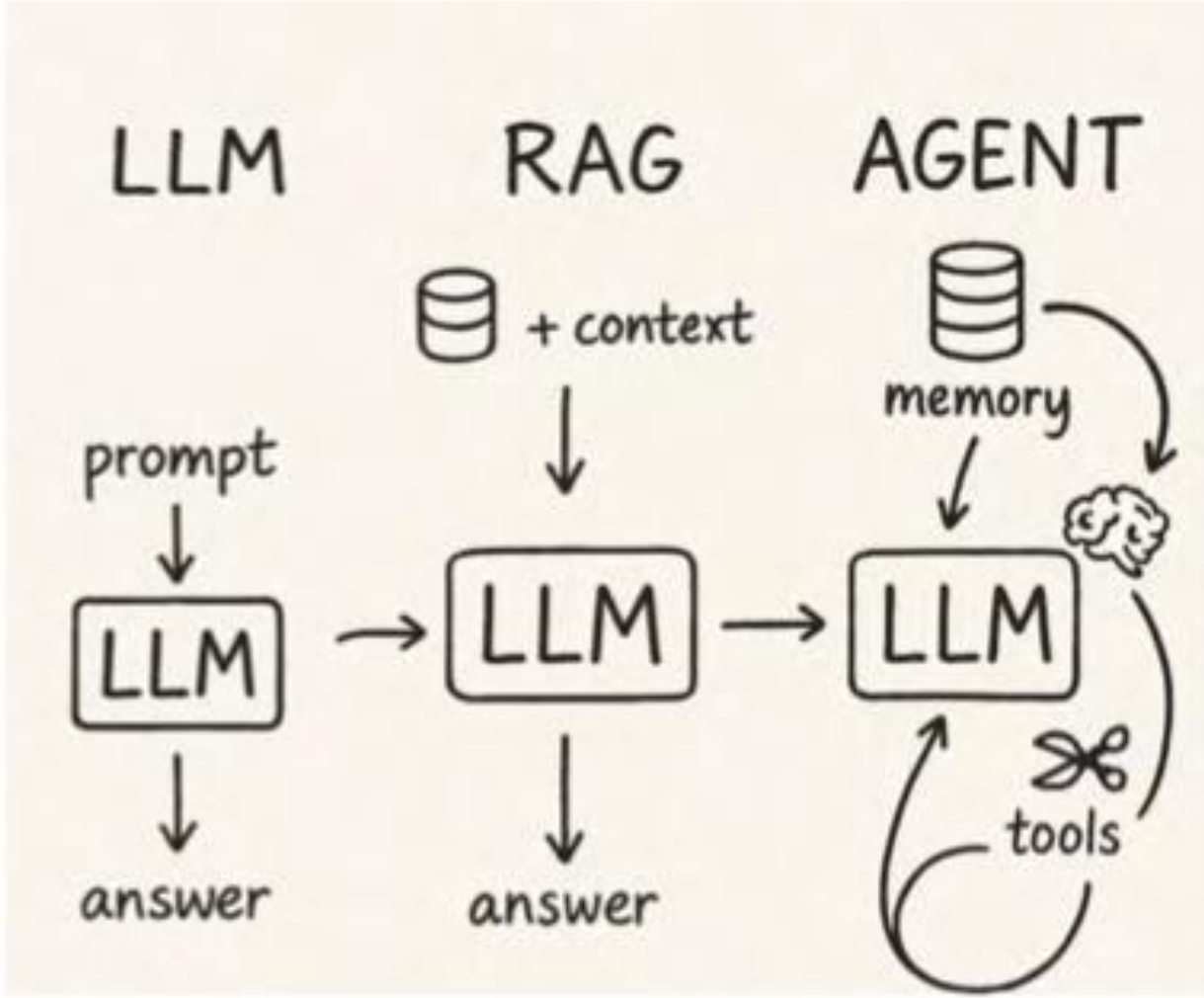
Advantages of RAG

- **RAG grounds responses in real-world retrieved data**

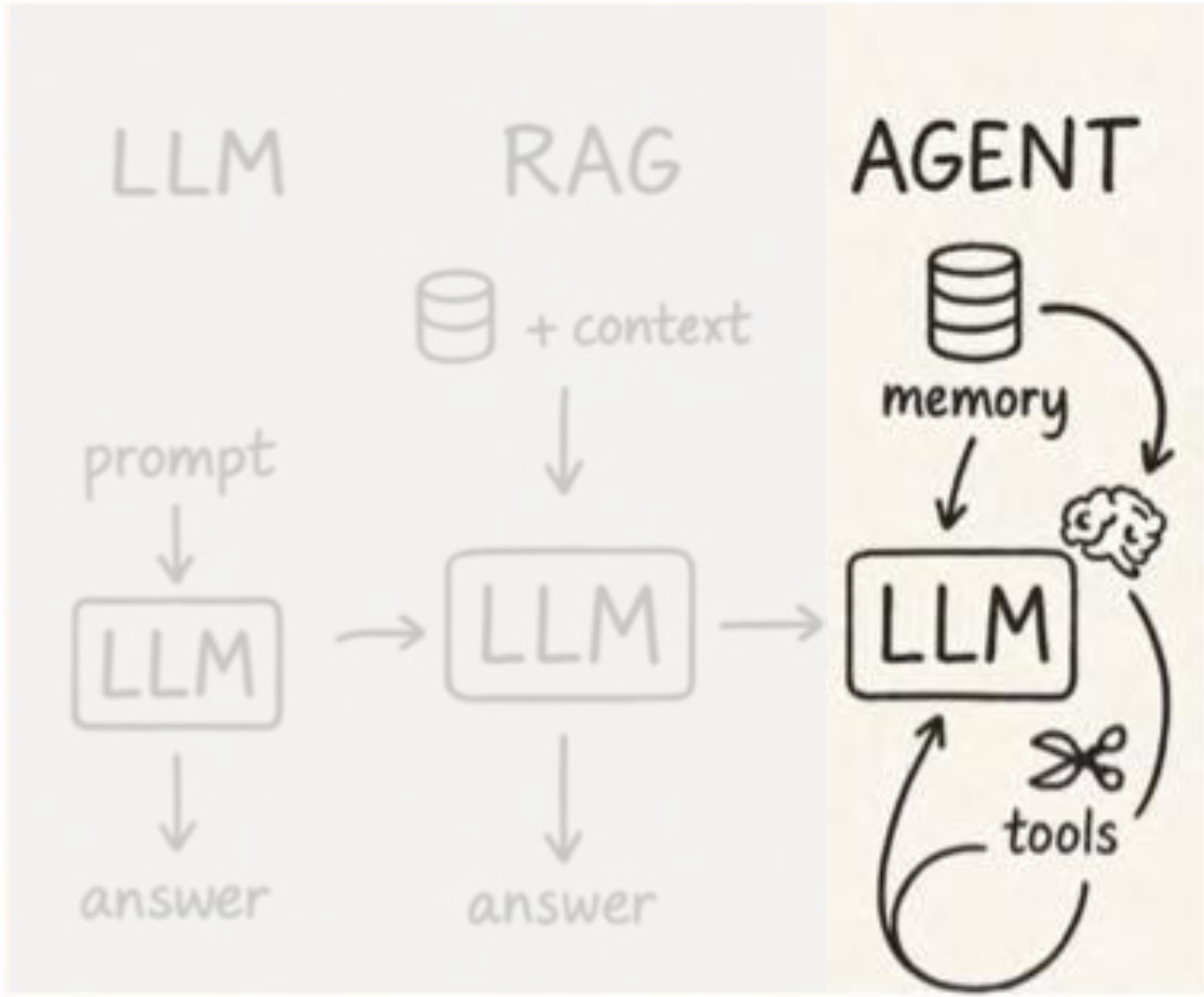
Query: "Does intent matter in a breach of contract?"

- **GPT Output:**
  - "A breach of contract only occurs when the contract is intentionally broken." (Incorrect—intent is not always required.)
- **RAG Output:**
  - "A breach of contract can occur when either party fails to perform their duties, regardless of intent, as specified under contract law and confirmed by legal standards." (Accurate, well-sourced, and fact-based!)

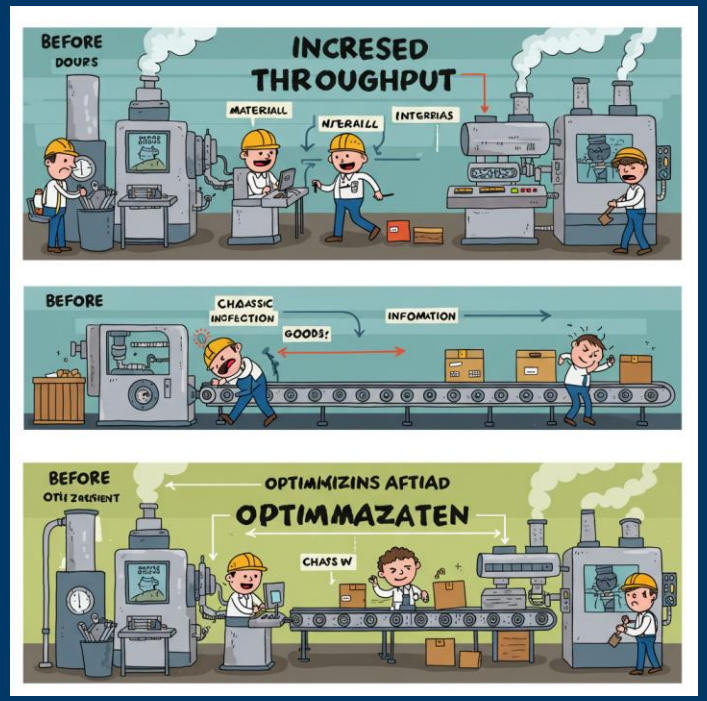
LLM – RAG – Agentic AI



LLM – RAG – Agentic AI



What are AI-Driven Improvements?



Increased Throughput

Reduced Inventory or Backlog

Cost Reduction

Requirements-Driven AI = Process Knowledge

- What is the current business process? Is it measured? Is it efficient?
- Where are there: backlogs, high inventory, danger/unsafe, high attrition, monotony?
- What are critical decisions that affect the business?
- How might we increase throughput/growth? Customer retention?
- What other pain points may exist?

Tools

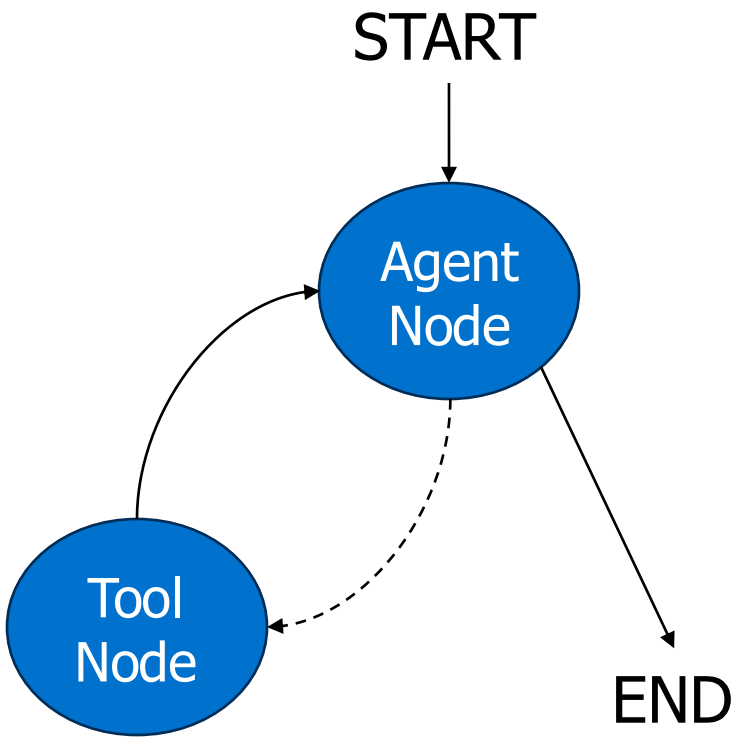
```python
def tool(a: type, b: type) -> type:
    """Description of the tool

    Args:
        a: first argument
        b: second argument
    """
    # Do some things

    return result
```

- External functions or API integrations that empower agents to perform actions
- Tool integrations are what make Agents useful
- Many existing tools in LangChain

The Agent Pattern



- Encapsulates **autonomous behavior** by integrating LLM and tools into a single unit.
- Acts as an intelligent actor that processes inputs, executes actions, and responds dynamically.
- Typically implemented as a function node that orchestrates various tasks within a graph.

Key Takeaways

- The Agent design patterns are the primary patterns for tool use. They allow for the execution of tools and the Agent to continue to use tools as necessary to answer the user prompt.
- Graphs can often be combinations of design patterns. For example, using a router with a planner agent to different execution agents that are agent patterns.
- Increasing the amount of autonomy decreases the predictability of outcomes.

Your Turn